

2026-08-10 更新:三信号对照实验、 λ 训练集子集重新验证、Steering 方案调研

一、gate 三种信号的对照实验

目标是把"仅用局部信号 Δh " (gate v2)、"仅用全局信号 h "、"两者特征级融合" (gate v3, 主方法) 放在同一套协议下做控制变量对比, 完整清单见 `gate_v3_main_method_report.md` 附录二。

1.1 gate 自身判分能力 (离线分类, dev 集) ——已完成

三者层选择、池化协议完全一致 ($j=0,1 \rightarrow$ 第15层, $j=2,3 \rightarrow$ 第23层), 唯一变量是输入特征:

变体	特征	dev AUC	overall F1	overall TPR
仅局部信号 (gate v2)	Δh_j	0.9221	0.7785	0.8456
仅全局信号 (新增)	h_j	0.9339	0.7939	0.876
特征级融合 (gate v3)	$h_j \oplus \Delta h_j$	0.9357	0.793	0.8838

仅用 h_j 单独训练的判分能力已经非常接近融合模型, overall F1 甚至略高于融合模型 (差距在噪声 范围内), 说明 h_j 是比 Δh_j 更强的单一信号。

1.2 端到端效果——进行中与规划

统一配置: MuSiQue dev 全量 2417 条、`bart_decompose`、 $\lambda=0.25$ 、不叠加检索指令前缀等额外改进, 隔离"信号来源"这一个变量。

组别	状态	说明
A. 纯余弦相似度 (无 gate)	进行中	musique 已完成, hotpot/2wiki 跑在 cuda1/cuda3 上
B. 仅局部信号 (gate v2)	已完成	recall@1=0.6627 recall@3=0.8256 chain@1=0.4357 chain@3=0.6756 EM=0.4100 F1=0.5045
C. 仅全局信号 (h-only)	规划中	依赖 1.1 节已产出的 artifact, 接入端到端脚本后即可跑
D. 特征级融合 (gate v3)	规划中	端到端答案指标已有, 真实 recall@3/chain@3 待补 (复用 A 组所用脚本的 oracle 计算逻辑, 在剪枝前的完整候选池上算)

A 组 musique 部分已跑完, 真实无 gate 基线:

	recall@1	recall@3	chain@1	chain@3	EM	F1
baseline (musique)	0.6003	0.7611	0.3335	0.5710	0.3757	0.4652

hotpot、2wiki 两个数据集跑完后补齐这张表, 随后接入 C 组 (h-only 端到端) 和 D 组的真实 recall@3/chain@3。另外, rawprefix + BGE 检索指令前缀这套改进目前只在 musique 上验证过,

2WikiMultihopQA、HotpotQA 上的验证也在规划中，跟 A/C/D 的端到端补充放在一起跑。

二、 λ 重新验证：训练集子集上的扫描

目前所有 λ 扫描 (0.10–0.60) 都是直接在 MuSiQue **dev 全量** 上做的，挑出来的 $\lambda=0.50$ 也是在 同一份 dev 数据上验证效果——用来选参数的数据和用来报告最终效果的数据是同一份。计划从 MuSiQue **训练集** 里抽一个子集，单独在这个子集上重新扫一遍 λ ，选出的 λ 再拿到 dev 上验证，作为对现有 $\lambda=0.50$ 选择的一次交叉检验。

预计步骤：

1. 从 `data/raw/musique/musique_ans_v1.0_train.jsonl` 里按 K 分层抽样出一个子集（规模待定，参考 dev 全量 2417 条的量级）。
2. 用 `decompose/bart/decompose_to_nl.py` 对这个子集跑一版 BART decompose 推理（目前 `data/decompose/musique/bart/` 下只有 `dev_nl.jsonl`，训练集子集还没有对应的分解结果，需要先补上这一步）。
3. 在这个子集上用 `run_retrieval_exp_wavefront_gate_v3.py` 重复现有的 λ 扫描流程 (0.10–0.60，间隔 0.05)。
4. 对比训练子集选出的最优 λ 与 dev 全量选出的 0.50 是否一致；若不一致，用训练子集选出的 λ 在 dev 全量上重新跑一次端到端，作为最终确认。

三、Steering 方案调研——已完成

完整报告见 `update_doc/steering_research_report.md`。核心结论：

- **方向向量不需要重新训练**：直接把现有 gate (v2/h-only/v3 三者之一) 的 `LogisticRegression.coef_` 反投影回 4096 维隐藏空间即可得到一个可用的 steering 方向，三种 artifact 的字典 key 和反投影公式已在报告 3.1 节给出。
- **核心工程约束**：gate 打分用的是 HF `transformers`（可以直接挂 hook，但不负责生成文本），真正生成推理链路文本的是 vLLM（v1 引擎的 CUDA Graph/PagedAttention 不支持标准 forward hook）——这个约束决定了落地路径的优先级。
- **建议的第一步（几小时、不需要 GPU 生成、零工程改动）**：把方向向量加到已知“应判定为异常”的负例隐藏状态上，看 gate 自己的判分是否系统性地往“正常”方向移动。这一步不成立的话，后续生成侧的工程投入没有意义，应作为启动整个方向前的第一个检查点。
- **后续路径**：验证方向有效后，优先做“仅对 gate 判定风险较高的跳，生成临时切到 HF `transformers.generate()` 并注入方向向量”这条最小改动路径；vLLM 内部挂 hook、训练专用 SAE/CAA 方向都是更重的选项，只有在前一步证明方向本身有效之后才考虑投入。
- 强度设计上借鉴了 2025 年“自适应强度 steering”的思路，用 gate 本身已有的连续概率决定干预强弱，不用额外训练自适应模块。